

EC-341 Digital System Design

Experiment # 1

Introduction to Verilog and ModelSim

1. Objective/s:

- Introduction to Verilog Coding
- Overview of ModelSim Environment.
- Lab Tasks.

2. Verilog

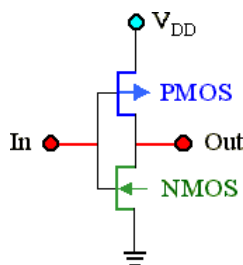
2.1. Introduction to Verilog

Verilog is a HDL (Hardware Description Language). It is used to model and simulate digital electronic circuits. Once a design is simulated, tested and ready for 'tape-out' to the fab, it can be synthesized to produce gate level designs that are then translated to physical design. In some ways, you can think of Verilog HDL as the code that represents the highest level abstraction of electronic circuits. It is different from general purpose programming languages in that it is specifically used to model hardware. So in Verilog you have the ability to specify registers, wires, gates, clock, etc. It is very useful when you have hardware specs on paper, and you want to simulate and test it first before synthesizing the circuit, thus saving time and money. The language is also pretty simple, only requires knowledge of digital logic, and it has syntax similar to C.

2.2. Verilog Abstraction Level

2.2.1. Switch (Transistor) Level

The lowest level of abstraction for a digital HDL would be the **switch level**, which refers to the ability to describe the circuit as a netlist of transistor switches. A more detailed modeling scheme that can catch some additional electrical problems when transistors are used in this way. Now, little-used because circuits generally aren't built this way.



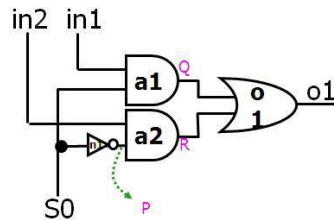
```
module cmos_inv (In, Out); // module declaration
input In;
output Out;
supply1 vdd;
supply0 gnd;
    pmos p1(Out, vdd, In); // pmos instantiation
    nmos n1(Out, gnd, In); // nmoc instantiation
endmodule
```

Fig. CMOS Inverter

Verilog Code

2.2.2. Gate (Structural) Level

A slightly higher level of abstraction would be the **gate level**, which refers to the ability to describe the circuit as a netlist of primitive logic gates and functions. The gates have one scalar output and multiple scalar inputs.



```

module mux2_1 (o1,in1,in2,S0); //module
output o1;
input in1,in2,S0;
and a1 (Q, in1, S0); //and instantiation
not n1 (P,S0);      // not instantiation
and a2 (R, in2, P); // and instantiation
or o1 (o1, Q, R); // or instantiation
endmodule

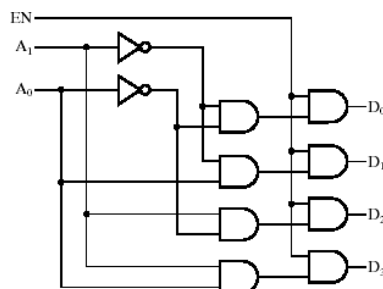
```

Fig. Multiplexer 2:1

Verilog Code

2.2.3. Data Flow Level

For small circuits, the gate level modeling approach works very well because the number of gates is limited and designer can instantiate and connects every gate individually. However in complex design the number of gates is very large. Thus implementing the function at a level higher than gate level is good choice. Dataflow modeling has become a popular design approach as logic synthesis tools have become sophisticated. This approach allows the designer to concentrate on optimizing the circuit in terms of data flow.



```

module deco2_4 (EN, A0, A1, D0, D1, D2,
D3); //module
input EN, A0, A1;
output D0, D1, D2, D3;
assign D0 =(EN & ~A1 & ~A0);
assign D1 =(EN & ~A1 & A0);
assign D2 =(EN & A1 & ~A0);
assign D3 =(EN & A1 & A0);
endmodule

```

Fig. Line Decoder (2-4)

Verilog Code

2.2.4. Functional and Algorithmic Level (Behavioral Modeling)

Verilog provides the designer the ability to describe the design functionality in an algorithmic manner. In other words the designer describes the **behavior** of the circuit. The abstraction in this modeling is as simple as writing the logic in **C** language. Verilog behavioral models contain procedural statements that control the simulation and manipulate variables of the data types

previously described. The activity starts at the control constructs initial and always. Each initial statement and each always statement starts a separate activity flow. All of the activity flows are concurrent, allowing the user to model the inherent concurrence of hardware.

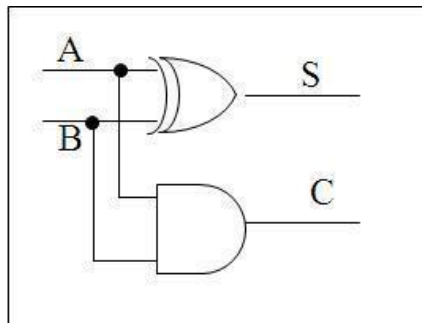


Fig. Binary counter(8-bit)

```

module half_add (S, C, A, B);
  input A, B;
  output S, C;
  reg S, C;
  always @ (A or B)
  begin
    S = A ^ B;
    C = A & B;
  end
end

```

Verilog Code

2.2.5. Register transfer Level (RTL Modelling)

Describes a system by the flow of data and controls signals within and between functional blocks. It's a behavioral design concept in which you make HDL models of registered circuits and how signals interact between them such as memories, flip flops, latches, shift registers, and so on. RTL is a combination of Behavioral and Data flow modeling which should be synthesizable. RTL description is more complex and less technology dependent than behavior hardware description.

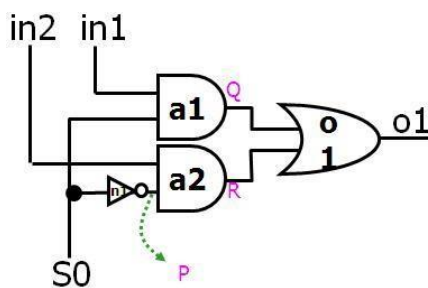


Fig. Multiplexer 2:1

```

module mux2to1 (in1,in2, S0, o1);
  input in1,in2,S0;
  output o1;
  always_comb begin
    case (sel)
      1'b0 : o1 = in1;
      1'b1 : o1 = in2;
      default : o1 = 1'bx;
    endcase
  end
end

```

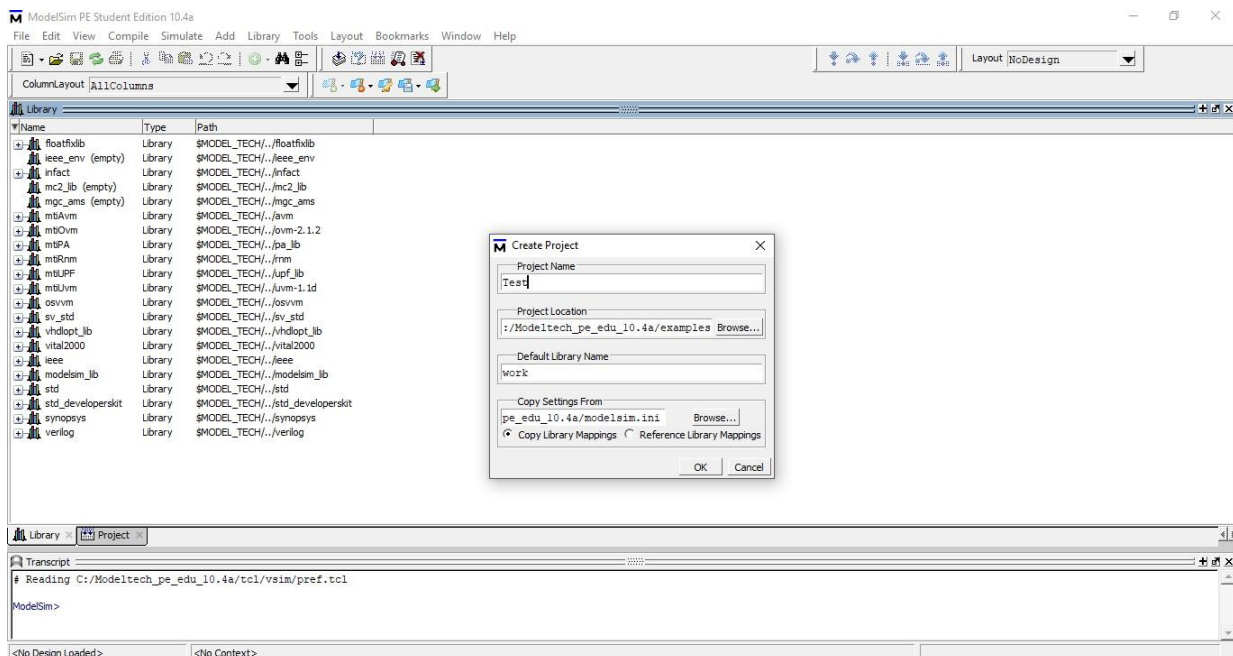
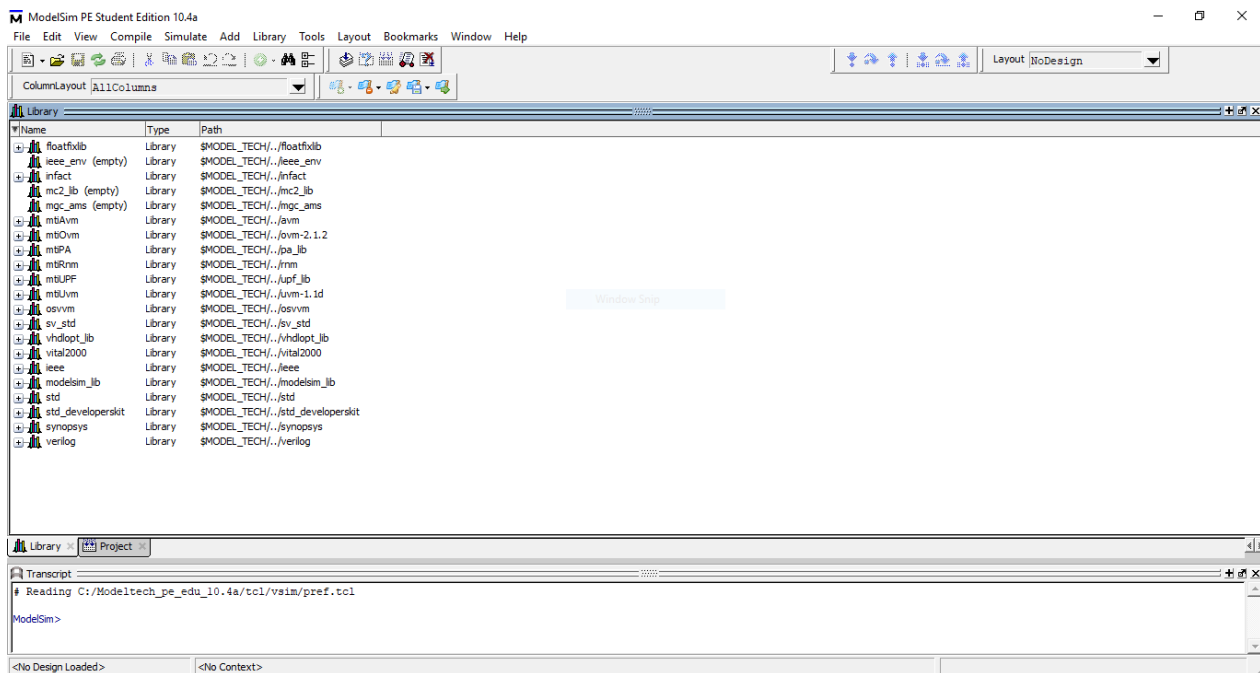
Verilog Code

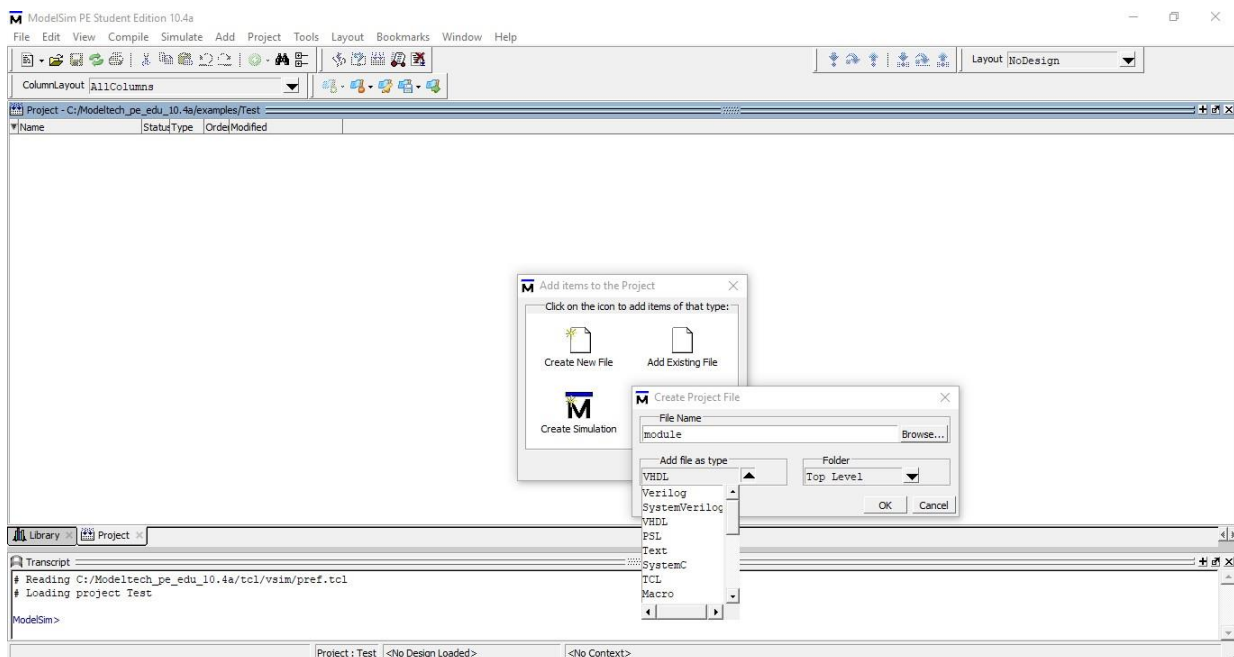
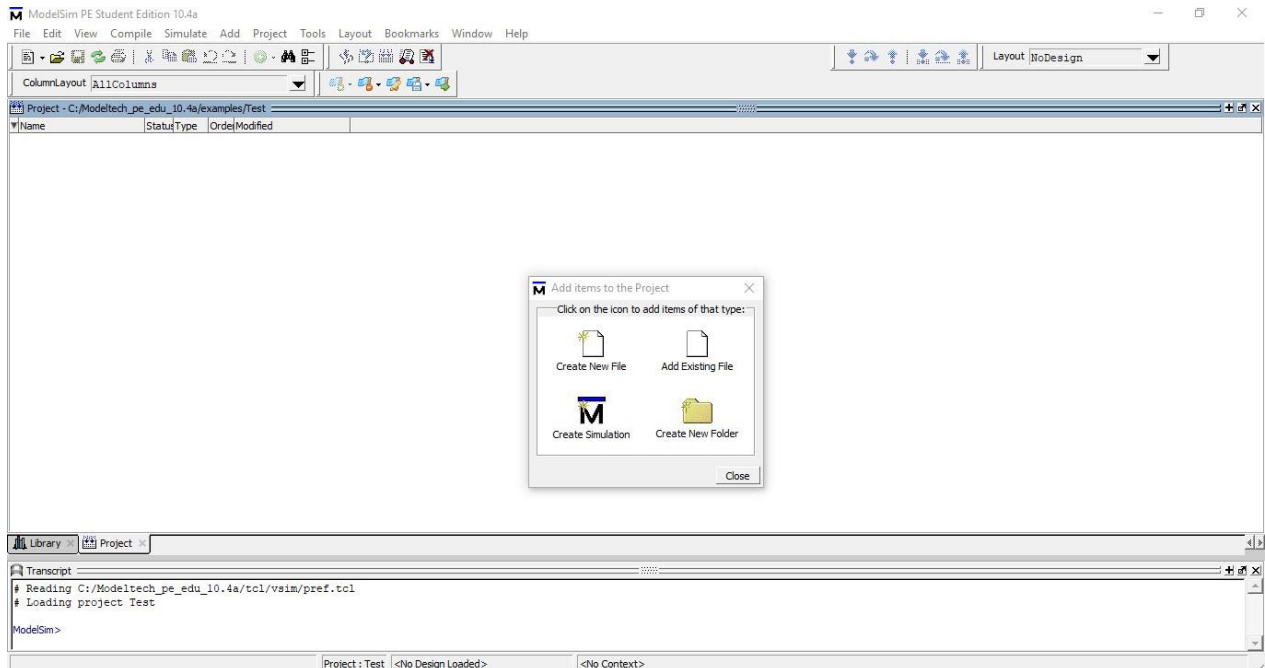
3. Overview of ModelSim Environment

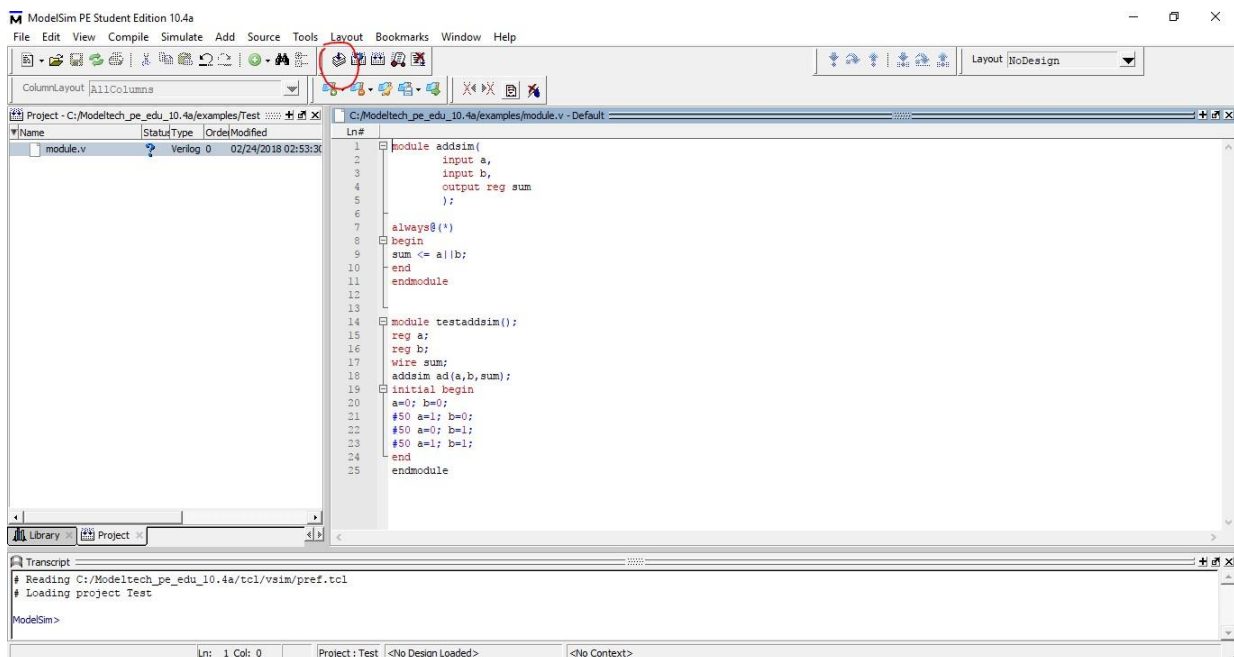
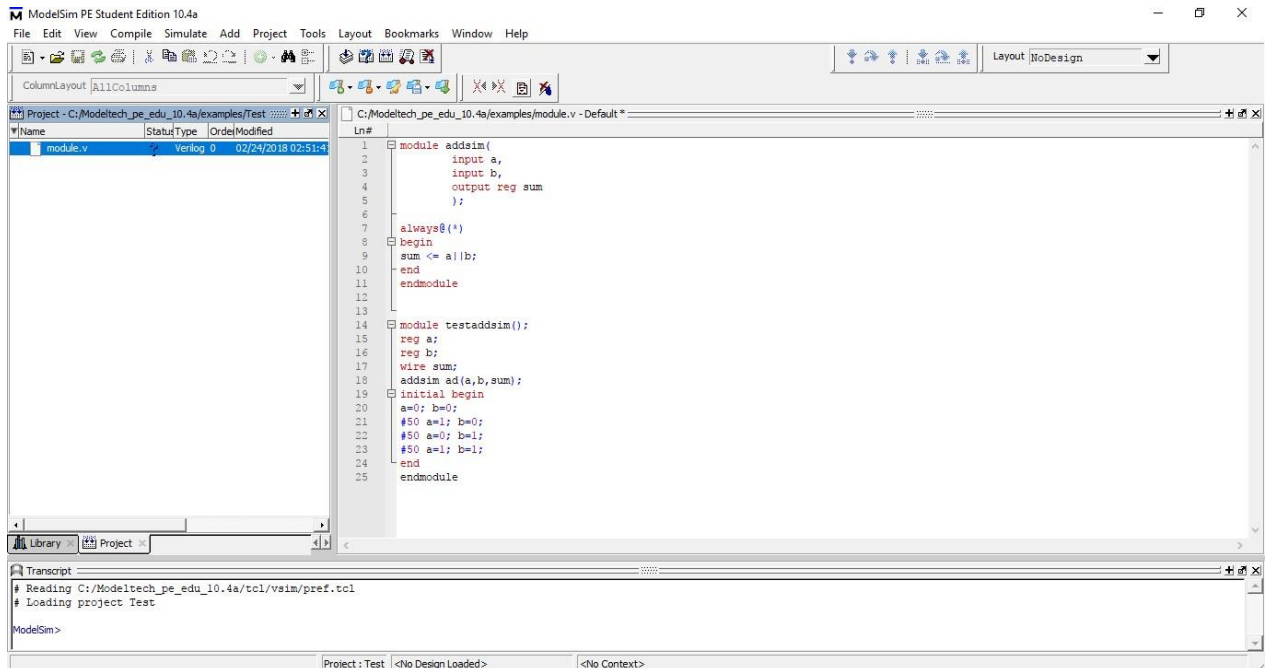
ModelSim is an easy-to-use yet versatile VHDL/(System)Verilog/SystemC simulator by Mentor Graphics. It supports behavioral, register transfer level, and gate-level modeling. ModelSim supports all platforms (i.e. Linux, Solaris and Windows) and many others too. On Linux and Solaris platforms ModelSim can be found preinstalled (see [Linux/Mustatikli](#)). Windows users, however, must install it by themselves.

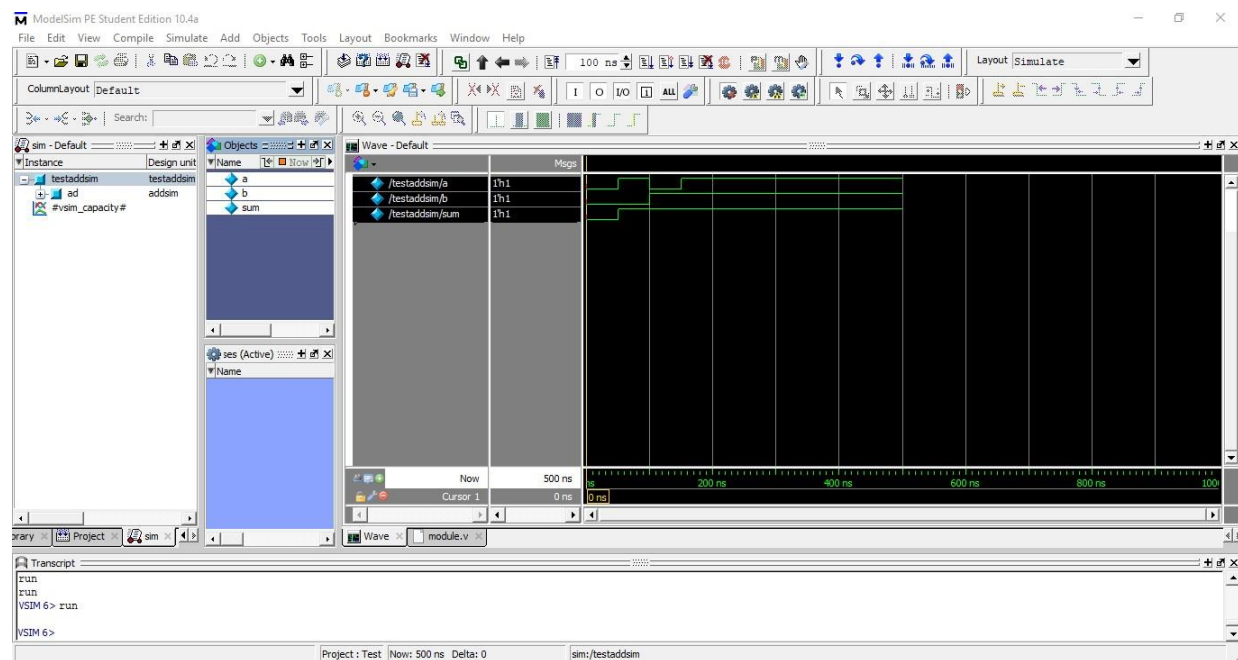
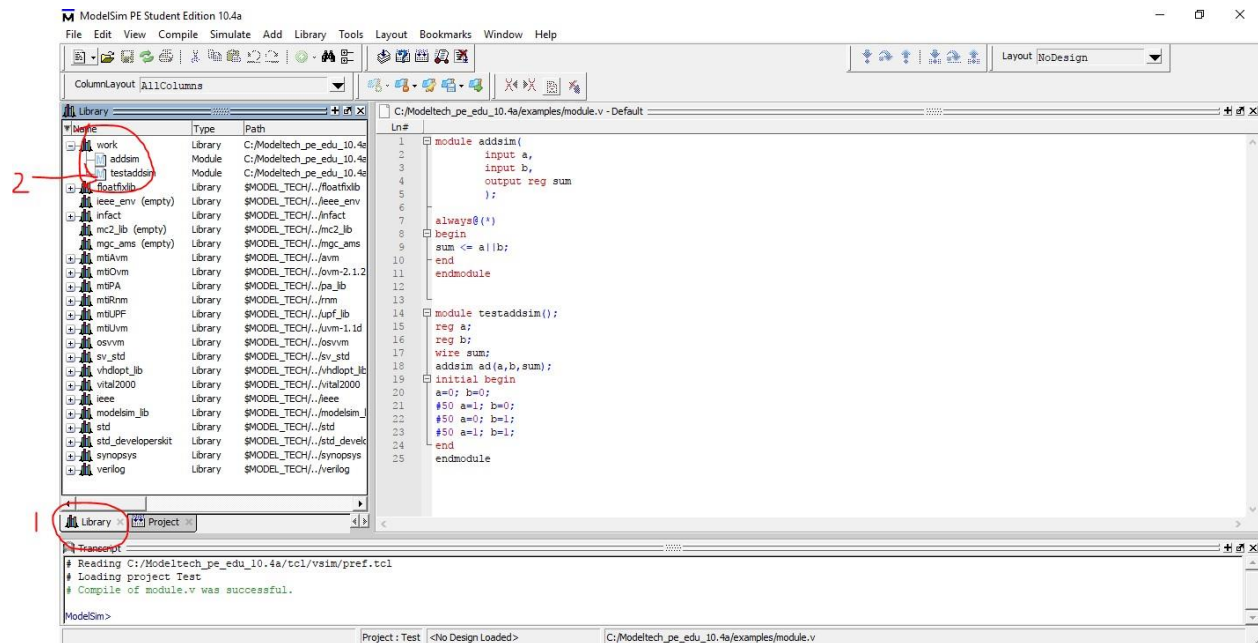
3.1. Getting Started With ModelSim

To Start a new Project and to Simulate the code, follow the following sets









4. Lab Tasks

1. *Write* a Gate Level Verilog code to find 1's complement of three bit numbers.
2. *Write* a Gate Level Verilog code for Half Adder and write code for test bench as well. Simulate the Adder for different set of inputs. Attach waveform screenshots and Verilog code
3. *Write* code for above task for Full Adder as well.